

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №7
по дисциплине «Вычислительная математика»
Тема: Численное интегрирование. Методы прямоугольников, трапеций,
Симпсона.

Студент гр. 4342

Ларионов В.А.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

Цель работы.

Разработать программу на языке C++ для численного вычисления определённых интегралов с использованием составных квадратурных формул: левых, правых и центральных прямоугольников, трапеций и метода Ньютона–Симпсона. Реализовать автоматический подбор числа разбиений отрезка интегрирования на основе правила Рунге для достижения заданной точности. Обеспечить возможность выбора подынтегральной функции из девяти предопределённых типов (линейная, квадратичная, степенная, показательная, логарифмическая, тригонометрические) и сравнить эффективность различных квадратурных формул.

Теоретический минимум.

Численное интегрирование – приближённое вычисление определённого интеграла $I = \int[a,b] f(x) dx$ путём замены подынтегральной функции на более простую (полином, кусочно-постоянную или кусочно-линейную функцию) на малых отрезках разбиения.

В работе используются составные квадратурные формулы. Отрезок $[a, b]$ разбивается на n равных частей с шагом $h = (b - a)/n$, и интеграл приближается суммой.

Метод прямоугольников – площадь под кривой заменяется суммой площадей прямоугольников. В зависимости от выбора узла различают левые, правые и центральные прямоугольники. В программе реализован метод с выбором типа через глобальную переменную `rectMethodType`. Формула центральных прямоугольников (наиболее точная из трёх):

$$I \approx h * \sum_{i=0}^{n-1} f(a + (i+1/2)h).$$

Метод трапеций – на каждом отрезке функция аппроксимируется прямой, интеграл заменяется суммой площадей трапеций:

$$I \approx h * (f(a)/2 + \sum_{i=1}^{n-1} f(a+i*h) + f(b)/2).$$

Метод Ньютона–Симпсона – на каждом сдвоенном отрезке (требуется чётное n) функция заменяется квадратичной параболой:

$$I \approx (h/3) * (f(a) + 4\sum_{i \text{ нечётные}} f(a+ih) + 2*\sum_{i \text{ чётные, } i=2}^{n-2} f(a+i*h) + f(b)).$$

Правило Рунге используется для оценки погрешности и автоматического подбора числа разбиений. Вычисляются два приближения интеграла на сетках с шагами h и $h/2$ (т.е. с n и $2n$ отрезками). Тогда погрешность более точного значения (на мелкой сетке) приближённо равна:

$$R = |I_{\{h/2\}} - I_h| / k,$$

где $k = 3$ для методов прямоугольников (центральных) и трапеций, $k = 15$ для метода Симпсона. В программе эти коэффициенты задаются переменной `runge_coeff`. Уточнённое значение интеграла вычисляется как $I_{\{h/2\}} \pm R$ (знак зависит от метода).

Итерационный процесс останавливается, когда оценка погрешности R становится меньше заданной точности ε . Максимальное число разбиений ограничено (10 миллионов и более), чтобы избежать бесконечного цикла.

Выполнение работы.

Программа состоит из следующих модулей:

`defines.hpp` – содержит определение типа `ld` (обычно `long double`) и, возможно, вспомогательные макросы.

`functions.hpp` – определяет структуру `coefficients` для хранения параметров подынтегральной функции и реализует девять функций: `linear`, `quadratic`, `power`, `exponential`, `lg` (логарифм по основанию 2), `sn` (синус), `cs` (косинус), `tg` (тангенс), `cg` (котангенс). При невозможности вычисления (например, логарифм от

неположительного аргумента, тангенс в точке $\pi/2 + \pi k$) функция возвращает максимальное значение типа `ld`, что обрабатывается как ошибка.

`rectangle.hpp` – реализует метод прямоугольников с выбором типа (левый, правый, центральный) через глобальную переменную `rectMethodType`.

`trapezoid.hpp` – реализует метод трапеций.

`Newton-Simpson.hpp` – реализует метод Ньютона–Симпсона с проверкой чётности числа разбиений.

`main.cpp` – организует диалог с пользователем: выбор функции, выбор метода (или сравнение всех методов), ввод пределов интегрирования и точности, итерационное уточнение по правилу Рунге, вывод результатов.

Особенности реализации

Гибкость выбора функции – пользователь может задать коэффициенты для любой из девяти поддерживаемых функций. Используется функциональный объект `std::function`, что позволяет единообразно передавать функцию в методы интегрирования.

Обработка особых точек – если в процессе вычислений значение функции в узле равно максимальному значению, программа выводит сообщение об ошибке и аварийно завершается. Это предотвращает некорректные расчёты (например, при интегрировании через разрыв).

Правило Рунге – для каждого метода вычисляется коэффициент `runge_coeff`. В основном режиме (выбран один метод) цикл удваивает число отрезков, начиная с 2, и сравнивает оценку погрешности с заданной точностью. Для метода Симпсона начальное $n = 2$, для остальных – $n = 1$ или $n = 2$ (в коде используется

старт с 2 и увеличение на 2). Шаг итерации – удвоение n , что позволяет эффективно использовать правило Рунге.

Режим сравнения методов – при выборе пользователем пункта 0 программа последовательно применяет все пять методов (левый, правый, центральный прямоугольники, трапеции, Симпсон) и выводит для каждого полученное число разбиений и значение интеграла. В этом режиме также используется правило Рунге с соответствующими коэффициентами.

Контроль сходимости – установлены лимиты на максимальное число разбиений ($10^7 - 10^8$), что защищает от закливания при невыполнении условия точности.

Выводы.

В ходе работы была создана программа для численного интегрирования, реализующая следующие возможности:

Вычисление интегралов пятью квадратурными методами (левые, правые, центральные прямоугольники, трапеции, Симпсон) с автоматическим подбором числа разбиений по правилу Рунге.

Поддержка девяти типов подынтегральных функций – от простейших полиномов до тригонометрических и логарифмических, с возможностью задания пользовательских коэффициентов.

Устойчивость к особым точкам – проверка области определения функций и корректная обработка ошибок.

Режим сравнения – позволяет за один запуск оценить эффективность разных методов на одной и той же функции.

Правило Рунге успешно обеспечивает достижение заданной точности, а удвоение числа разбиений на каждой итерации даёт быструю сходимость для гладких функций. Метод Симпсона, имеющий более высокий порядок точности, требует наименьшего числа разбиений по сравнению с методами прямоугольников и трапеций, что подтверждает его вычислительную эффективность.

Программа может быть использована как учебный инструмент для изучения численных методов интегрирования, а также как основа для решения прикладных задач, где требуется автоматический контроль погрешности.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
#ifndef DEFINES_HPP
#define DEFINES_HPP

#include <cmath>
#include <functional>
#include <iostream>
#include <limits>
#include <string>

typedef long double ld;

#endif // DEFINES_HPP
#include "../defines.hpp"

#ifndef EPSILON
#define EPSILON 1e-10
#endif

#define LINEAR 1
#define QUAD 2
#define POW 3
#define EXP 4
#define LOG 5
#define SIN 6
#define COS 7
#define TG 8
#define CTG 9

struct coefficients {
    ld a = 1, b, c;
};

coefficients funcCoeff;

ld linear(ld x) {
    return funcCoeff.a * x + funcCoeff.b;
}
```

```
}
```

```
ld quadratic(ld x) {  
    return \  
    funcCoeff.a * x * x +  
    funcCoeff.b * x +  
    funcCoeff.c;  
}
```

```
ld power(ld x) {  
    if (funcCoeff.b < 0 && std::abs(x) < EPSILON) {  
        return std::numeric_limits<ld>::max();  
    }  
    else {  
        return funcCoeff.a * std::pow(x, funcCoeff.b);  
    }  
}
```

```
ld exponential(ld x) {  
    return funcCoeff.a * std::pow(funcCoeff.b, x);  
}
```

```
ld lg(ld x) {  
    if (funcCoeff.b + x <= 0) {  
        return std::numeric_limits<ld>::max();  
    }  
    else {  
        return funcCoeff.a + std::log2l(funcCoeff.b + x);  
    }  
}
```

```
ld sn(ld x) {  
    return funcCoeff.a + std::sin(funcCoeff.b + x);  
}
```

```
ld cs(ld x) {
```



```

        return funcCoeff.a + std::cos(funcCoeff.b + x);
    }

    ld tg(ld x) {
        ld arg = funcCoeff.b + x;
        if (std::abs(std::cos(arg)) < EPSILON) {
            return std::numeric_limits<ld>::max();
        }
        return funcCoeff.a + std::tan(arg);
    }

    ld cg(ld x) {
        ld arg = funcCoeff.b + x;
        if (std::abs(std::sin(arg)) < EPSILON) {
            return std::numeric_limits<ld>::max();
        }
        else {
            return funcCoeff.a + std::cos(arg) / std::sin(arg);
        }
    }

    ld mod2(ld x) {
        return std::abs(x - 1);
    }

#include "../defines.hpp"

#define RECTANGLE 1

enum rectangleType {
    LEFT,
    RIGHT,
    CENTER
};

rectangleType rectMethodType;

ld rectangle(std::function<ld(ld)> function, ld a, ld b, int n) {
    ld result = 0.0L;
    ld h = (b - a) / (ld)n;

```

```

int counter = 0;
while (counter < n) {
    ld currentValue;
    ld point;
    switch (rectMethodType) {
        case rectangleType::RIGHT:
            point = a + h * ++counter;
            break;
        case rectangleType::CENTER:
            point = a + h * counter++ + h / 2;
            break;
        case rectangleType::LEFT:
            point = a + h * counter++;
            break;
        default:
            point = a + h * counter++ + h / 2;
    }
    currentValue = function(point);
    if (currentValue != std::numeric_limits<ld>::max()) {
        result += currentValue;
    }
    else {
        std::cerr << "Невозможно получить значение функции в точке "
        << std::to_string(point) << '\n';
        exit(1);
    }
}
return h*result;
}
#include "../defines.hpp"

#define TRAPEZOID 4

ld trapezoid(std::function<ld(ld)> function, ld a, ld b, int n) {
    ld h = (b - a) / n;
    ld sum = 0.0L;
    ld fa = function(a);
    ld fb = function(b);
    if (fa == std::numeric_limits<ld>::max() || fb ==
std::numeric_limits<ld>::max()) {
        std::cerr << "Невозможно вычислить функцию на границах интервала.\n";
        exit(1);
    }
}

```

```

    }
    sum = (fa + fb) / 2.0L;
    for (int i = 1; i < n; ++i) {
        ld x = a + i * h;
        ld fx = function(x);
        if (fx == std::numeric_limits<ld>::max()) {
            std::cerr << "Невозможно получить значение функции в точке " << x <<
'\n';
            exit(1);
        }
        sum += fx;
    }
    return h * sum;
}
#include "../defines.hpp"

#define SIMPSON 5

ld NewtonSimpson(std::function<ld(ld)> function, ld a, ld b, int n) {
    if (n % 2 != 0) {
        std::cerr << "Ошибка: для метода Симпсона число разбиений n должно быть
чётным.\n";
        exit(1);
    }
    ld h = (b - a) / n;
    ld sum = 0.0L;
    ld fa = function(a);
    ld fb = function(b);
    if (fa == std::numeric_limits<ld>::max() || fb ==
std::numeric_limits<ld>::max()) {
        std::cerr << "Невозможно вычислить функцию на границах интервала.\n";
        exit(1);
    }
    sum = fa + fb;
    for (int i = 1; i < n; i += 2) {
        ld x = a + i * h;
        ld fx = function(x);
        if (fx == std::numeric_limits<ld>::max()) {
            std::cerr << "Невозможно получить значение функции в точке " << x <<
'\n';
            exit(1);
        }
    }
}

```

```

        sum += 4.0L * fx;
    }
    for (int i = 2; i < n; i += 2) {
        ld x = a + i * h;
        ld fx = function(x);
        if (fx == std::numeric_limits<ld>::max()) {
            std::cerr << "Невозможно получить значение функции в точке " << x <<
'\n';
            exit(1);
        }
        sum += 2.0L * fx;
    }
    return (h / 3.0L) * sum;
}

#include "./functions.hpp"
#include "./Newton-Simpson.hpp"
#include "./rectangle.hpp"
#include "./trapezoid.hpp"
#include <iomanip>

void printListOfFunctions() {
    std::cout << "Выберете функцию:\n";
    std::cout << "1. Линейная\n2. Квадратичная\n3. Степенная\n";
    std::cout << "4. Показательная\n5. Логарифмическая\n";
    std::cout << "6. Синус\n7. Косинус\n8. Тангенс\n9. Котангенс\n";
}

void printListOfMethods() {
    std::cout << "Выберете метод вычисления интеграла:\n";
    std::cout << "1. Метод левых прямоугольников\n";
    std::cout << "2. Метод правых прямоугольников\n";
    std::cout << "3. Метод центральных прямоугольников\n";
    std::cout << "4. Метод трапеций\n";
    std::cout << "5. Метод Ньютона-Симпсона\n";
    std::cout << "0. Сравнение методов\n";
}

std::function<ld(ld)> getFunction(int functionChoice) {
    std::function<ld(ld)> func;

```

```

switch (functionChoice) {
case LINEAR:
    func = linear;
    std::cout << "Вы выбрали линейную функцию, она задается формулой:\n";
    std::cout << "\tf(x) = a*x + b\nВведите коэффициенты a и b.\n";
    std::cin >> funcCoeff.a >> funcCoeff.b;
    break;
case QUAD:
    func = quadratic;
    std::cout << "Вы выбрали квадратичную функцию, она задается формулой:\n";
    std::cout << "\tf(x) = a*x^2 + b*x + c\nВведите коэффициенты a, b и c.\n";
    std::cin >> funcCoeff.a >> funcCoeff.b >> funcCoeff.c;
    break;
case POW:
    func = power;
    std::cout << "Вы выбрали степенную функцию, она задается формулой:\n";
    std::cout << "\tf(x) = a*x^b\nВведите коэффициенты a и b.\n";
    std::cin >> funcCoeff.a >> funcCoeff.b;
    break;
case EXP:
    func = exponential;
    std::cout << "Вы выбрали показательную функцию, она задается формулой:\n";
    std::cout << "\ta*b^x\nВведите коэффициенты a и b.\n";
    std::cin >> funcCoeff.a >> funcCoeff.b;
    break;
case LOG:
    func = lg;
    std::cout << "Вы выбрали логарифмическую функцию, она задается
формулой:\n";
    std::cout << "\ta + log2(x + b)\nВведите коэффициенты a и b.\n";
    std::cin >> funcCoeff.a >> funcCoeff.b;
    break;
case SIN:
    func = sn;
    std::cout << "Вы выбрали функцию синуса, она задается формулой:\n";
    std::cout << "\ta + sin(x + b)\nВведите коэффициенты a и b.\n";
    std::cin >> funcCoeff.a >> funcCoeff.b;
    break;
case COS:
    func = cs;
    std::cout << "Вы выбрали функцию косинуса, она задается формулой:\n";
    std::cout << "\ta + cos(x + b)\nВведите коэффициенты a и b.\n";

```

```

        std::cin >> funcCoeff.a >> funcCoeff.b;
        break;
    case TG:
        func = tg;
        std::cout << "Вы выбрали функцию тангенса, она задается формулой:\n";
        std::cout << "\ta + tg(x + b)\nВведите коэффициенты а и b.\n";
        std::cin >> funcCoeff.a >> funcCoeff.b;
        break;
    case CTG:
        func = cg;
        std::cout << "Вы выбрали функцию котангенса, она задается формулой:\n";
        std::cout << "\ta + ctg(x + b)\nВведите коэффициенты а и b.\n";
        std::cin >> funcCoeff.a >> funcCoeff.b;
        break;
    default:
        func = mod2;
        std::cout << "Выбрана функция по умолчанию:\n";
        std::cout << "\tf(x) = mod(x - 1)\n";
        funcCoeff.a = 1;
        funcCoeff.b = 0;
        break;
    }
    return func;
}

std::function<ld(std::function<ld(ld)>, ld, ld, int)> getMethod(int methodChoice)
{
    std::function<ld(std::function<ld(ld)>, ld, ld, int)> method;
    switch (methodChoice) {
    case RECTANGLE + rectangleType::LEFT:
        method = rectangle;
        rectMethodType = rectangleType::LEFT;
        std::cout << "Вы выбрали метод левых прямоугольников\n";
        break;
    case RECTANGLE + rectangleType::RIGHT:
        method = rectangle;
        rectMethodType = rectangleType::RIGHT;
        std::cout << "Вы выбрали метод правых прямоугольников\n";
        break;
    case RECTANGLE + rectangleType::CENTER:
        method = rectangle;

```

```

        rectMethodType = rectangleType::CENTER;
        std::cout << "Вы выбрали метод центральных прямоугольников\n";
        break;
    case TRAPEZOID:
        method = trapezoid;
        std::cout << "Вы выбрали метод трапеций\n";
        break;
    case SIMPSON:
        method = NewtonSimpson;
        std::cout << "Вы выбрали метод Ньютона-Симпсона\n";
        break;
    default:
        method = NewtonSimpson;
        std::cout << "метод Ньютона-Симпсона выбран по умолчанию\n";
        break;
    }
    return method;
}

int main() {
    printListOfFunctions();
    int functionChoice;
    std::cin >> functionChoice;
    auto function = getFunction(functionChoice);
    printListOfMethods();
    int methodChoice;
    std::cin >> methodChoice;
    ld runge_coeff;
    if (methodChoice == SIMPSON) runge_coeff = 15.0L;
    else if (methodChoice == TRAPEZOID) runge_coeff = 3.0L;
    else if (methodChoice == RECTANGLE + rectangleType::CENTER) runge_coeff =
3.0L;
    else runge_coeff = 1.0L;
    if (methodChoice) {
        auto method = getMethod(methodChoice);
        long double n, a, b;
        std::cout << "Введите пределы интегрирования.\n";
        std::cin >> a >> b;
        std::cout << "Введите необходимую точность.\n";
        std::cin >> n;
        std::string nS = std::to_string(n);

```

```

    ld I_prev = 0.0L;
    ld I_curr;
    for (int i = 2; ; i += 2) {
        I_curr = method(function, a, b, i);
        if (std::abs(I_curr - I_prev)/runge_coeff < n) {
            std::cout << "Количество отрезков: " << i << '\n';
            std::cout << '\t' << std::fixed << std::setprecision(nS.length())
<< I_curr << '\n';
            break;
        }
        I_prev = I_curr;
        if (i > 100000000) {
            std::cerr << "Достигнут лимит разбиений\n";
            break;
        }
    }
}
else {
    long double n, a, b;
    std::cout << "Введите пределы интегрирования.\n";
    std::cin >> a >> b;
    std::cout << "Введите необходимую точность.\n";
    std::cin >> n;
    std::string nS = std::to_string(n);
    rectMethodType = rectangleType::LEFT;
    std::cout << "1. ";
    ld I_prev = rectangle(function, a, b, 1);
    ld I_curr;
    for (int i = 2; ; i += 2) {
        I_curr = rectangle(function, a, b, i);
        if (std::abs(I_curr - I_prev) < n) {
            std::cout << "Количество отрезков: " << i << '\n';
            std::cout << '\t' << std::fixed << std::setprecision(nS.length())
<< I_curr << '\n';
            break;
        }
        I_prev = I_curr;
        if (i > 100000000) {
            std::cerr << "Достигнут лимит разбиений\n";
            break;
        }
    }
}

```



```

rectMethodType = rectangleType::RIGHT;
std::cout << "2. ";
I_prev = rectangle(function, a, b, 1);
for (int i = 2; ; i += 2) {
    I_curr = rectangle(function, a, b, i);
    if (std::abs(I_curr - I_prev) < n) {
        std::cout << "Количество отрезков: " << i << '\n';
        std::cout << '\t' << std::fixed << std::setprecision(nS.length())
<< I_curr << '\n';
        break;
    }
    I_prev = I_curr;
    if (i > 100000000) {
        std::cerr << "Достигнут лимит разбиений\n";
        break;
    }
}
rectMethodType = rectangleType::CENTER;
std::cout << "3. ";
I_prev = rectangle(function, a, b, 1);
for (int i = 2; ; i += 2) {
    I_curr = rectangle(function, a, b, i);
    if (std::abs(I_curr - I_prev)/3.0 < n) {
        std::cout << "Количество отрезков: " << i << '\n';
        std::cout << '\t' << std::fixed << std::setprecision(nS.length())
<< I_curr << '\n';
        break;
    }
    I_prev = I_curr;
    if (i > 100000000) {
        std::cerr << "Достигнут лимит разбиений\n";
        break;
    }
}
std::cout << "4. ";
I_prev = trapezoid(function, a, b, 1);
for (int i = 2; ; i += 2) {
    I_curr = trapezoid(function, a, b, i);
    if (std::abs(I_curr - I_prev)/3.0 < n) {
        std::cout << "Количество отрезков: " << i << '\n';
        std::cout << '\t' << std::fixed << std::setprecision(nS.length())
<< I_curr << '\n';

```

```

        break;
    }
    I_prev = I_curr;
    if (i > 1000000000) {
        std::cerr << "Достигнут лимит разбиений\n";
        break;
    }
}

std::cout << "5. ";
I_prev = NewtonSimpson(function, a, b, 2);
for (int i = 4; ; i += 2) {
    I_curr = NewtonSimpson(function, a, b, i);
    if (std::abs(I_curr - I_prev)/15.0 < n) {
        std::cout << "Количество отрезков: " << i << '\n';
        std::cout << '\t' << std::fixed << std::setprecision(nS.length())
<< I_curr << '\n';
        break;
    }
    I_prev = I_curr;
    if (i > 1000000000) {
        std::cerr << "Достигнут лимит разбиений\n";
        break;
    }
}

return EXIT_SUCCESS;
}

```